

Module 4 Concurrency

Module:4	Concurrency	8 hours
Inter-process communication, Synchronization - Implementing synchronization primitives (Peterson's solution, Bakery algorithm, synchronization hardware) - Semaphores – Classical synchronization problems, Monitors: Solution to Dining Philosophers problem – IPC in Unix, Multiprocessors and Locking - Scalable Locks - Lock-free coordination.		

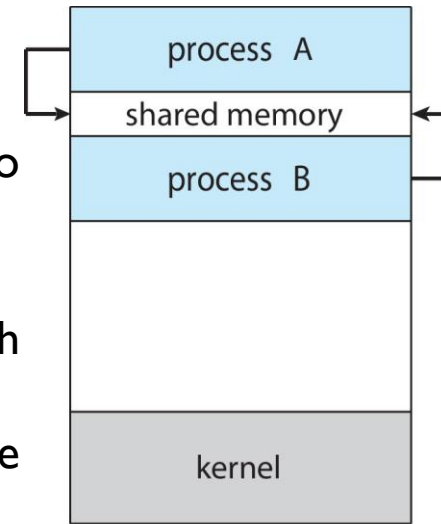
Inter process Communication

- Processes executing concurrently in the operating system may be either **independent processes or cooperating processes**.
- A process is **independent** if it does not share data with any other processes executing in the system.
- A process is **cooperating** if it can affect or be affected by the other processes executing in the system, including sharing data.
- Reasons for cooperating processes:
 - Information sharing
 - Computation speedup
 - Modularity
 - Convenience
- Cooperating processes need **interprocess communication (IPC)** that will allow them to exchange data, i.e that is, send data to and receive data from each other.
- Two fundamental models of IPC
 - **Shared memory**
 - **Message passing**

Inter process Communication

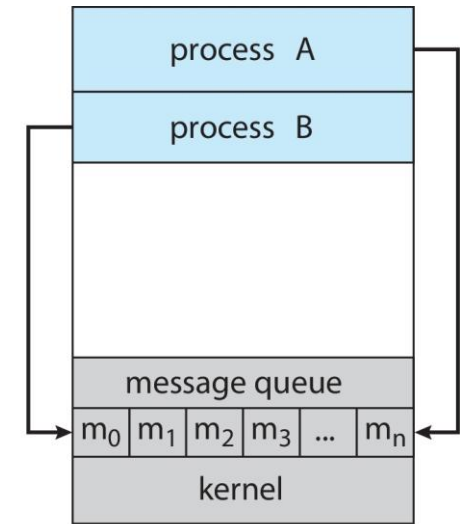
Shared memory

- In the shared-memory model, a region of memory that is shared by the cooperating processes is established.
- Processes can then exchange information by reading and writing data to the shared region.
- Shared memory can be faster than message passing.
- In shared-memory systems, system calls are required only to establish shared memory regions.
- Once shared memory is established, all accesses are treated as routine memory accesses, and no assistance from the kernel is required.
- Instead of sending messages through the OS every time, processes communicate by directly **reading and writing the same memory area.**



(a)

(a) Shared memory.



(b)

(b) Message passing.

Message passing

- communication takes place by means of messages exchanged between the cooperating processes.
- Message passing is also easier to implement in a distributed system.
- Since message-passing systems are typically implemented using system calls and thus require the more time-consuming task of kernel intervention

IPC in Shared-Memory Systems

- Interprocess communication using shared memory requires communicating processes to establish a region of shared memory.
- Typically, a shared-memory region resides in the address space of the process creating the shared-memory segment.
- Other processes that wish to communicate using this shared-memory segment must attach it to their address space.
- Recall that, normally, the operating system tries to prevent one process from accessing another process's memory.
- Shared memory requires that two or more processes agree to remove this restriction. They can then exchange information by reading and writing data in the shared areas.
- The form of the data and the location are determined by these processes and are not under the operating system's control.
- The processes are also responsible for ensuring that they are not writing to the same location simultaneously.

Producer consumer Problem

- The **Producer–Consumer Problem** is a classic **process synchronization problem** in Operating Systems.
- In this problem, there are two processes:
 - ✓ The **Producer** generates data and places it into a shared buffer.
 - ✓ The **Consumer** takes data from that shared buffer and processes it.
- For example: Producer → Shared Buffer → Consumer
- If the buffer is full, the **producer must wait** before adding more data. If the buffer is empty, the **consumer must wait** until new data is produced.
- The main issue is that both processes access the same shared memory, so proper synchronization is required to avoid **race conditions, buffer overflow, and buffer underflow**.
- Usually, synchronization is achieved using **semaphores or mutex locks**.

The Producer–Consumer Problem deals with coordinating a producer that adds data to a shared buffer and a consumer that removes data from it safely.

Bounded-Buffer – Shared-Memory Solution

One solution to the producer–consumer problem uses shared memory.

- To allow producer and consumer processes to run concurrently, use a buffer of items that can be filled by the producer and emptied by the consumer.
- This buffer will reside in a region of memory that is shared by the producer and consumer processes.
- A producer can produce one item while the consumer is consuming another item.
- The producer and consumer must be synchronized, so that the consumer does not try to consume an item that has not yet been produced.

Two types of buffers can be used.

- The **unbounded buffer** places no practical limit on the size of the buffer. The consumer may have to wait for new items, but the producer can always produce new items.
- The **bounded buffer** assumes a fixed buffer size. In this case, the consumer must wait if the buffer is empty, and the producer must wait if the buffer is full.

Bounded-Buffer – Shared-Memory Solution

Illustrates interprocess communication using shared memory.

- The following variables reside in a region of memory shared by the producer and consumer processes:
- **Shared data**

```
#define BUFFER_SIZE 10
typedef struct {
    ...
} item;

item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

- The shared buffer is implemented as a circular array with two logical pointers: **in and out**.
- The variable in points to the next free position in the buffer;
- Out points to the first full position in the buffer.
- **The buffer is empty when $in == out$;**
- **The buffer is full when $((in + 1) \% BUFFER_SIZE) == out$**
- Solution is correct, but can only use **$BUFFER_SIZE - 1$** elements.

Bounded-Buffer – Shared-Memory Solution

- The producer process has a local variable next produced in which the new item to be produced is stored.
- The consumer process has a local variable next consumed in which the item to be consumed is stored.
- This scheme allows at most BUFFER SIZE – 1 items in the buffer at the same time.

```
item next_produced;
```

```
while (true) {  
    /* produce an item in next produced */  
    while (((in + 1) % BUFFER_SIZE) == out)  
        ; /* do nothing */  
    buffer[in] = next_produced;  
    in = (in + 1) % BUFFER_SIZE;  
}
```

```
item next_consumed;
```

```
while (true) {  
    while (in == out)  
        ; /* do nothing */  
    next_consumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
  
    /* consume the item in next consumed */  
}
```

- One issue this illustration does not address concerns the situation in which both the producer process and the consumer process attempt to access the shared buffer concurrently.

IPC in Message-Passing Systems

- Another way to achieve the same effect is for the operating system to provide the means for cooperating processes to communicate with each other via a message-passing facility.
- Message passing provides a mechanism to allow processes to communicate and to synchronize their actions without sharing the same address space.
- It is particularly useful in a distributed environment, where the communicating processes may reside on different computers connected by a network.
- Message system – processes communicate with each other without resorting to shared variables
- IPC facility provides two operations:
 - send(message)
 - receive(message)
- The message size is either fixed or variable

Message Passing (Cont.)

- If processes P and Q wish to communicate, they need to:
 - Establish a **communication link** between them
 - Exchange messages via send/receive
- Implementation issues:
 - How are links established?
 - Can a link be associated with more than two processes?
 - How many links can there be between every pair of communicating processes?
 - What is the capacity of a link?
 - Is the size of a message that the link can accommodate fixed or variable?
 - Is a link unidirectional or bi-directional?

How are links established?

- Implementation of communication link
 - Physical:
 - Shared memory
 - Hardware bus
 - Network
 - Logical:
 - Direct or indirect
 - Synchronous or asynchronous
 - Automatic or explicit buffering

Message Passing (Cont.)

Direct Communication

- Processes must name each other explicitly:
 - **send** (*P*, *message*) – send a message to process *P*
 - **receive**(*Q*, *message*) – receive a message from process *Q*
- **Properties of communication link**
 - Links are established automatically
 - A link is associated with exactly one pair of communicating processes
 - Between each pair there exists exactly one link
 - The link may be unidirectional, but is usually bi-directional

Indirect Communication

- Messages are directed and received from mailboxes (also referred to as ports)
 - Each **mailbox** has a unique id
 - Processes can communicate only if they share a mailbox
- **Properties of communication link**
 - Link established only if processes share a common mailbox
 - A link may be associated with many processes
 - Each pair of processes may share several communication links
 - Link may be unidirectional or bi-directional

Message Passing (Cont.)

Indirect Communication

Operations

- create a new mailbox (port)
- send and receive messages through mailbox
- destroy a mailbox

Primitives are defined as:

send(A, *message*) – send a message to mailbox A

receive(A, *message*) – receive a message from mailbox A

Mailbox sharing

- P_1 , P_2 , and P_3 share mailbox A
- P_1 sends; P_2 and P_3 receive
- Who gets the message?

Solutions

- Allow a link to be associated with at most two processes
- Allow only one process at a time to execute a receive operation
- Allow the system to select arbitrarily the receiver. Sender is notified who the receiver was.

Synchronization

- Message passing may be either blocking or non-blocking
- **Blocking** is considered **synchronous**
 - ❑ **Blocking send** -- the sender is blocked until the message is received
 - ❑ **Blocking receive** -- the receiver is blocked until a message is available
- **Non-blocking** is considered **asynchronous**
 - ❑ **Non-blocking send** -- the sender sends the message and continue
 - ❑ **Non-blocking receive** -- the receiver receives:
 - A valid message, or
 - Null message
- Different combinations possible
 - If both send and receive are blocking, we have a **rendezvous**

Buffering

- Queue of messages attached to the link.
- Implemented in one of three ways
 1. Zero capacity – no messages are queued on a link. Sender must wait for receiver (rendezvous)
 2. Bounded capacity – finite length of n messages. Sender must wait if link full
 3. Unbounded capacity – infinite length, Sender never waits

Synchronization

- A synchronization problem occurs when two or more processes or threads execute concurrently and access shared data or shared resources, causing incorrect or unpredictable results if their execution is not properly coordinated.
- Process synchronization is the coordination of concurrent processes or threads so that shared resources are accessed in a controlled and correct manner.
- The main purpose of synchronization is to **prevent race conditions and maintain data consistency**.

- **Example**

Suppose two processes, P1 and P2, share a variable:

count = 5

Both processes execute:

count = count + 1

Ideally, after both execute:

count = 7

However, internally the operation may occur as:

Read count

Increment count

Write count

Consider this execution: Initial count = 5

P1 reads count → 5

P2 reads count → 5

P1 increments → 6

P2 increments → 6

P1 writes 6

P2 writes 6

Final result: count = 6

instead of: count = 7

This happens because both processes accessed the shared variable simultaneously.

This situation is called a **race condition**.

Synchronization

Critical Section

- The part of a program where a process accesses shared data or resources is called the **critical section**.
- Consider system of n processes $\{p_0, p_1, \dots, p_{n-1}\}$
- Each process has **critical section** segment of code
 - Process may be changing common variables, updating table, writing file, etc.
 - When one process in critical section, no other may be in its critical section

Critical section problem is to design protocol to solve this

- Each process must ask permission to enter critical section in **entry section**, may follow critical section with **exit section**, then **remainder section**
- For example: `count = count + 1;`
- If count is shared among multiple processes, this statement belongs to the critical section.
- A typical process can be represented as:

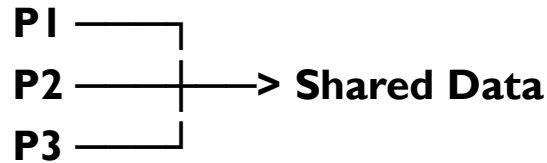
```
while (true) {  
    entry section  
    critical section  
    exit section  
    remainder section  
}
```

- The synchronization mechanism must ensure that when one process is executing its critical section, another conflicting process does not enter its critical section simultaneously.

Critical Section Problem

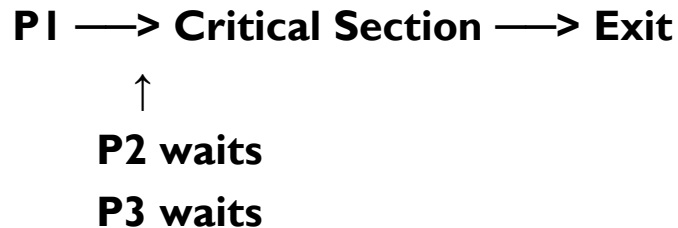
- Suppose there are processes: P1, P2, P3
- All of them want to access a shared resource.

Without synchronization:



- Multiple processes may modify the same data simultaneously.

With synchronization:



- Only the permitted process accesses the shared resource.

A correct solution to the critical-section problem should satisfy **three important conditions**:

1. **Mutual Exclusion** : Only one process can execute its critical section at a time.
2. **Progress** : If no process is inside the critical section, a process that wants to enter should eventually be allowed to enter.
3. **Bounded Waiting** : A process should not wait indefinitely for its turn to enter the critical section.

Why synchronization is required?

Synchronization is required whenever concurrent processes share resources such as:

- Shared variables
 - Shared memory
 - Files, Database records
 - Printers
 - Buffers , Data structures
 - I/O devices
- Without synchronization, problems such as **race conditions, inconsistent data, lost updates, and incorrect program execution** can occur.

Common synchronization mechanisms

- Operating systems provide several mechanisms to solve synchronization problems:

Mechanism	Purpose
Mutex	Allows only one process/thread to access a resource
Semaphore	Controls access to one or more shared resources
Monitor	High-level synchronization construct
Spinlock	Process repeatedly checks until a lock becomes available
Condition Variable	Allows processes to wait for a particular condition

Classical Approaches

Synchronization primitives are mechanisms used by an Operating System to ensure that **multiple processes or threads safely access shared resources**. Their main purpose is to solve the **critical-section problem** and prevent race conditions.

Three important classical approaches are:

- **Peterson's Solution**
- **Bakery Algorithm**
- **Hardware-based synchronization**

I. Peterson's Solution

- Peterson's solution is a **software-based synchronization algorithm for two processes**.
- Assume two processes: P0, P1
- It uses two shared variables:
 1. boolean flag[2];
 2. int turn;

Meaning of variables

- $\text{flag}[i] = \text{true}$ means process P_i wants to enter the critical section.
- turn indicates which process should be given priority when both processes want to enter simultaneously.

Petersons algorithm

Initially: flag[0] = false; flag[1] = false;

Algorithm for process P_i

Let:

i = current process

j = other process

Then:

```
do {  
    flag[i] = true;  
    turn = j;  
    while (flag[j] && turn == j)  
        ; // wait
```

/* Critical Section */

flag[i] = false;

/* Remainder Section */

```
} while (true);
```

For P₀:

```
do {  
    flag[0] = true;  
    turn = 1;  
    while (flag[1] && turn == 1)  
        ;  
    /* Critical Section */  
    flag[0] = false;  
    /* Remainder Section */  
} while (true);
```

For P₁:

```
do {  
    flag[1] = true;  
    turn = 0;  
    while (flag[0] && turn == 0)  
        ;  
    /* Critical Section */  
    flag[1] = false;  
    /* Remainder Section */  
} while (true);
```

Petersons algorithm

How Peterson's solution works?

Suppose both P0 and P1 want to enter the critical section.

Both set:

`flag[0] = true`

`flag[1] = true`

Then each process assigns turn to the other process.

Suppose the last assignment is:

`turn = 1`

Then:

P0 checks `flag[1] && turn == 1` → true, so P0 waits.

P1 checks `flag[0] && turn == 0` → false, so P1 enters the critical section.

When P1 exits:

`flag[1] = false`

Now P0 can enter.

Petersons Algorithm

Properties satisfied

- Peterson's algorithm satisfies the three conditions of the critical-section problem:
- **Mutual Exclusion:** Only one process can enter the critical section at a time.
- **Progress:** If the critical section is free, one of the waiting processes can enter.
- **Bounded Waiting:** A process cannot be postponed indefinitely.

Limitation

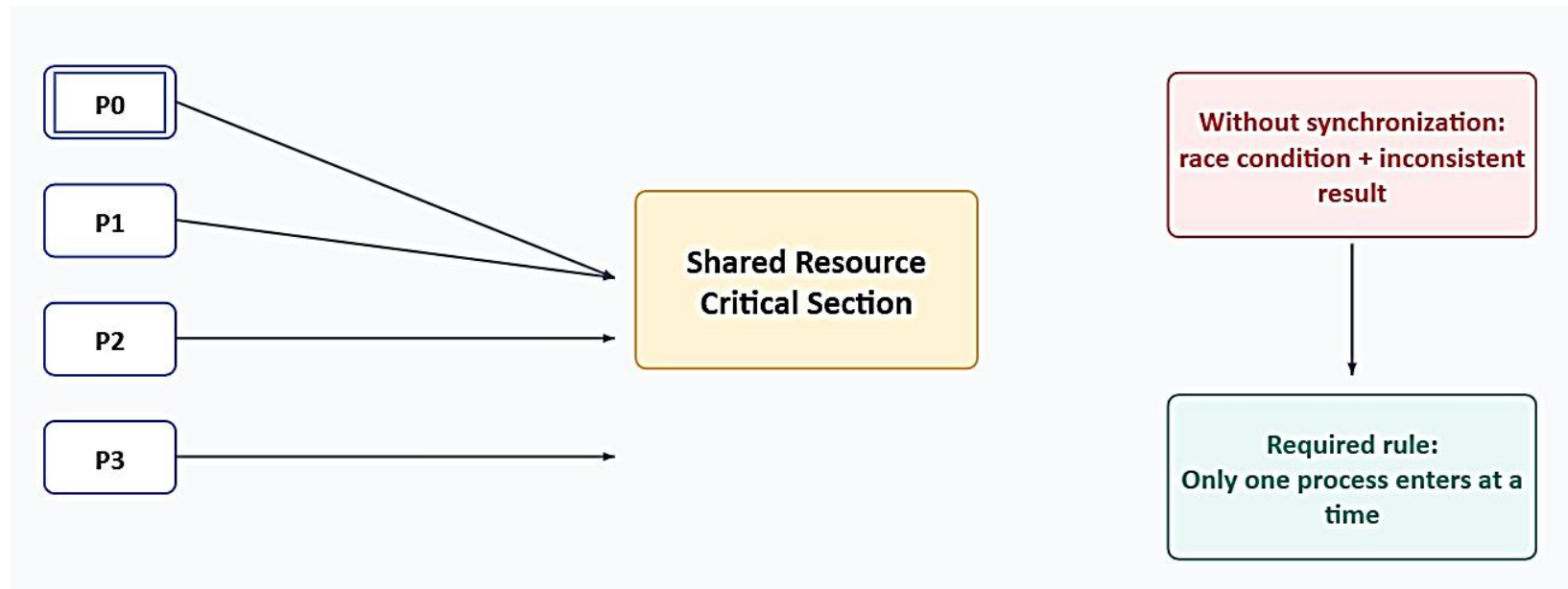
- Peterson's solution is mainly useful for understanding synchronization concepts.

Its major limitations are:

- Works **only for two processes**
- Uses **busy waiting**
- Modern CPUs and compilers may reorder memory operations, so the classical algorithm cannot be directly assumed correct without memory-order guarantees

2. Bakery Algorithm

- Bakery Algorithm is a token based software solution for process synchronization
- When multiple processes access the same shared resource concurrently, problems may occur.
- Possible problems:
 - Race condition
 - Data inconsistency
 - Lost updates
 - Multiple processes entering the critical section simultaneously
- **Goal:** Ensure that only one process enters the critical section at a time.

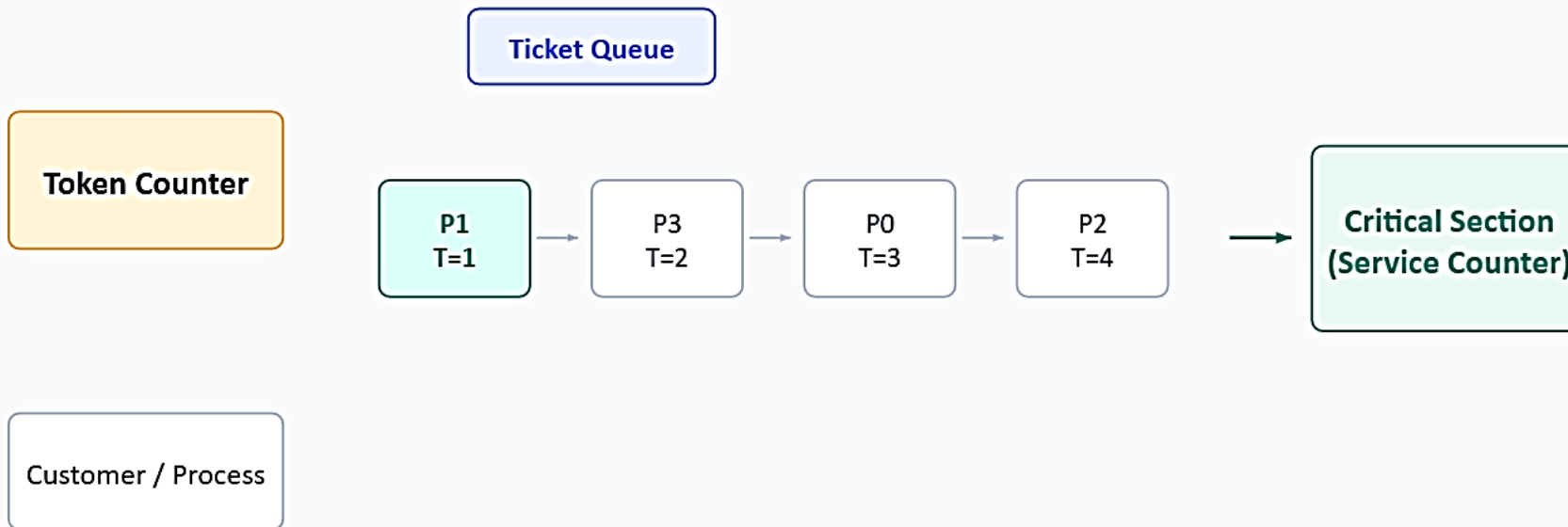


2. Bakery Algorithm

- The Bakery Algorithm works like a **token system in a bakery**.
- processes obtain numbers before entering the critical section.
- The process with the **smallest ticket number enters first**.

Bakery analogy: take a token and wait

Core intuition



Rule: smaller token number gets priority. If token numbers are equal, smaller process ID wins.

2. Bakery Algorithm

- The Bakery Algorithm uses two shared arrays:

- 1. choosing[i]** - Indicates whether process P_i is currently selecting its ticket.;
choosing[i] = true means P_i is choosing a ticket.
- 2. number[i]** - Stores the ticket number of process P_i .
number[i] = 5 means: P_i has ticket number 5.

choosing[i]

Indicates whether process P_i is currently selecting its ticket number.

number[i]

Stores the ticket number of P_i .
number[i] = 0 means P_i is not competing.

Process	choosing[]	number[]
P0	false	0
P1	false	0
P2	false	0
P3	false	0

Initial state: nobody wants to enter

2. Bakery Algorithm

- A ticket number of zero has a special meaning.
- `number[i] = 0` means: Process P_i is not currently interested in entering the critical section.

Example:

Process	Ticket
P0	2
P1	0
P2	4
P3	0

Here:

- P0 and P2 are competing
- P1 and P3 are not competing

Initially: for all processes.

- `choosing[i] = false`
- `number[i] = 0`

Process	choosing	number
P0	false	0
P1	false	0
P2	false	0
P3	false	0

2. Bakery Algorithm

Step 1: Process Starts Choosing a Ticket

- Suppose P0 wants to enter the critical section.
- It first executes: `choosing[0] = true;` // P0 has started selecting its ticket number.
- This prevents other processes from making a decision about P0 before P0 completes ticket selection.

Step 2: Selecting the Ticket Number

- P0 executes: `number[0] = 1 + max(number[0], number[1], ..., number[n-1]);`
- Suppose: `number = [0, 0, 0, 0]`
Maximum value: `max = 0`
Therefore: `number[0] = 1`, So: P0 → Ticket 1

Process	choosing	number
P0	false	1
P1	false	0
P2	false	4
P3	false	0

Step 3: Finish Choosing

- After obtaining the ticket:
`choosing[i] = false;` // Pi has completed ticket selection.
- Example: Before: `choosing[2] = true`
After: `choosing[2] = false`
`number[2] = 4`
- Now other processes can safely compare their ticket with P2.

2. Bakery Algorithm

Priority Rule

- Processes are ordered based on: **Smaller ticket number**
- If tickets are equal, **smaller process ID** [Lexicographic Ordering]

Example:

P0 → Ticket 4

P1 → Ticket 2

P2 → Ticket 5

P3 → Ticket 2

Order: P1 → P3 → P0 → P2

Why? $(2,1) < (2,3) < (4,0) < (5,2)$

Why Process ID Is Needed

Two processes may select the same ticket.

Example:

P1 → Ticket 5

P3 → Ticket 5

Both have equal ticket numbers.

So compare: (Ticket, Process ID)

P1: (5,1)

P3: (5,3)

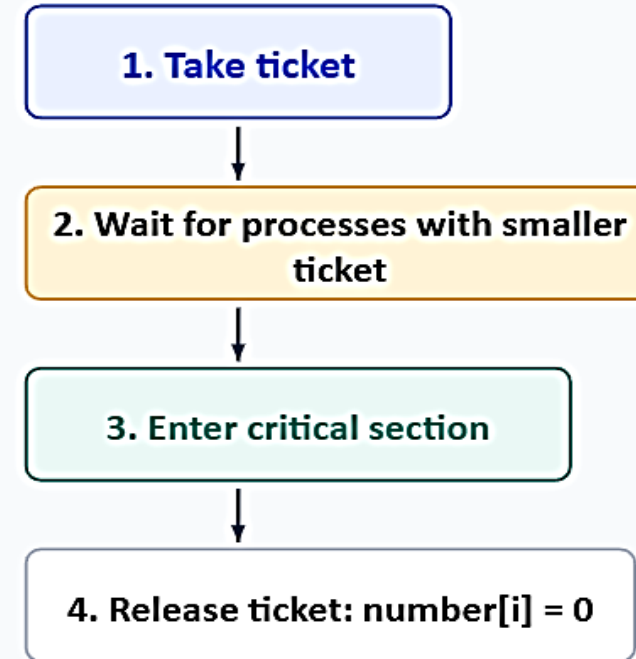
Since: $(5,1) < (5,3)$, P1 enters first.

2. Bakery Algorithm

Algorithm structure for process P_i

Entry → Critical → Exit

```
do {  
    choosing[i] = true;  
    number[i] = 1 + max(number[0..n-1]);  
    choosing[i] = false;  
  
    for each process  $P_j$ :  
        while (choosing[j]);  
        while (number[j] != 0 &&  
            (number[j], j) < (number[i], i));  
  
    /* critical section */  
  
    number[i] = 0;  
} while (true);
```



2. Bakery Algorithm

Waiting for Another Process to Finish Choosing

Each process checks:

while (choosing[j])

;

Meaning:

If P_j is currently choosing its ticket, wait until it completes the ticket-selection process.

Illustration:

P₁ → Choosing Ticket

|

| choosing[1] = true

|

P₂ → WAIT

Once: choosing[1] = false

P₂ can compare ticket numbers.

```
do {
    choosing[i] = true;
    number[i] = 1 + max(number[0..n-1]);
    choosing[i] = false;

    for each process Pj:
        while (choosing[j]);
        while (number[j] != 0 &&
              (number[j], j) < (number[i], i));

    /* critical section */

    number[i] = 0;
} while (true);
```

2. Bakery Algorithm - illustration

Ticket selection example

$\text{number}[i] = 1 + \max(\text{existing tickets})$

Current ticket array

P0	P1	P2	P3
1	4	0	2

P2 now wants to enter.

$$\max(1, 4, 0, 2) = 4$$



P2 gets ticket 5

Updated: number = [1, 4, 5, 2]

2. Bakery Algorithm

Waiting rule and tie-breaking

Who enters first?

Pi waits while:

```
number[j] != 0 &&
(number[j], j) < (number[i], i)
```

Meaning: Pj is competing and has higher priority than Pi.

Pi waits if:

- Pj is interested in entering, and
- Pj has higher priority

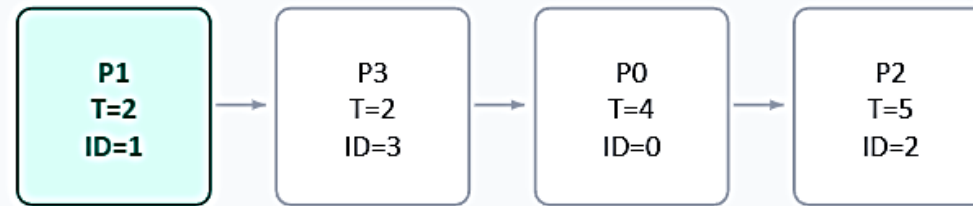
Higher priority means:

Smaller Ticket

or:

Same Ticket + Smaller Process ID

Example queue



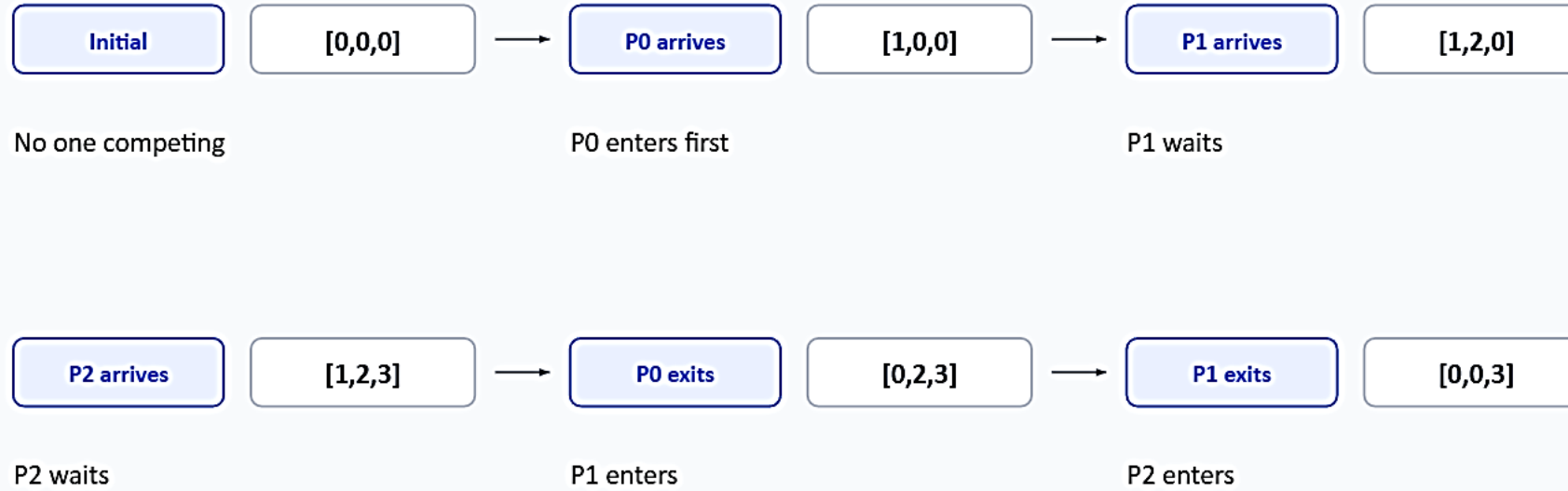
(2,1) < (2,3), so P1 goes before P3.

Priority = smaller ticket; if equal, smaller process ID.

2. Bakery Algorithm

Step-by-step execution with three processes

Illustrated run



The smallest non-zero ticket always gets access next.

2. Bakery Algorithm

Why the Bakery Algorithm works

Correctness properties

Mutual Exclusion

Two processes cannot both have the smallest ordered pair at the same time.

Progress

If the critical section is free, a competing process with the smallest ticket can proceed.

Bounded Waiting

A process waits only for processes with smaller tickets already ahead of it.

Therefore, starvation is prevented under the algorithm's theoretical assumptions.

The ordered pair (number[i], i) gives every competing process a unique priority.

2. Bakery Algorithm

Strengths

- Works for N processes
- Software-only concept
- Demonstrates fair ordering
- Avoids starvation conceptually

Limitations

- Busy waiting wastes CPU
- Large shared arrays
- Rarely used directly
- Modern systems prefer atomic hardware locks

Remember

Take ticket → wait for smaller tickets → enter critical section → set ticket to 0.

`while (choosing[j]);`

`while (...);`

The process continuously checks a condition.

This is called: **Busy Waiting**

Example:

P0 → Critical Section

P1 → Check → Check → Check → Check...

Disadvantage:

- CPU cycles are consumed while waiting

Bakery Algorithm = token-based mutual exclusion for multiple processes.

3. Hardware Support for Synchronization

- Software-based solutions are not guaranteed to work on modern computer architectures.
- The major hardware mechanisms provide support for solving the critical-section problem are:
 1. Disabling interrupts
 2. Atomic hardware instructions
 - a. Test-and-Set
 - b. Compare-and-Swap
 - c. Swap / Exchange
 3. Atomic variables

1. Disabling Interrupts

- On a **single-processor system**, the OS kernel can temporarily disable interrupts while executing a critical section.
- Since the processor cannot be interrupted, another process cannot take control of the CPU during this critical section.

Advantage: It is simple and effective for short kernel-level critical sections on a single CPU.

Limitation : It does not solve synchronization properly on multiprocessor systems.

3. Hardware Support for Synchronization

2. Atomic Hardware Instructions

- An **atomic instruction** executes as one indivisible operation.
- Hardware atomic instructions are one of the most important foundations of synchronization.
- Common instructions include:
 - Test-and-Set
 - Compare-and-Swap
 - Swap / Exchange
 - Fetch-and-Add

Semaphores

- A semaphore is an operating system synchronization mechanism used to control access to shared resources when multiple processes or threads are executing at the same time.
- A semaphore is a variable (a counter) that helps processes **decide whether they can enter a critical section or must wait.**
- A semaphore can ensure that only one process enters the critical section at a time.
- It prevents problems such as:
 - ❖ Two processes modifying the same data at the same time.
 - ❖ Race conditions.
 - ❖ Incorrect results due to simultaneous access.
- A semaphore maintains an integer value called the **semaphore value**. [Semaphore $S = 1$]
- The value indicates the number of available resources.
- $S = 1 \rightarrow$ One process can access the resource. the process has to wait.
- $S = 0 \rightarrow$ Resource is currently being used; others must wait.
- $S > 1 \rightarrow$ Multiple resources are available. a process may be allowed to use the resource.

Semaphores

- A semaphore is commonly used in situations where many processes or threads share the same resource, such as:
 - shared memory , files
 - printers
 - buffers
 - database connections
- A semaphore is essentially an **integer variable** that can be modified only through two atomic operations:
 1. **The wait()** operation is used when a process wants to use the resource.

```
wait(S)
{
    while(S <= 0)
        wait;
    S = S - 1;
}
```

- Check whether the resource is available.
- If available, decrease the semaphore value and enter.
- If not available, the process waits.

2. **The signal()** operation is used when the process finishes using the resource.

```
signal(S)
{
    S = S + 1;
}
```

- Release the resource.
- Increase the semaphore value.
- Allow another waiting process to proceed.

- The important point is that these operations are performed **atomically**, so two processes cannot modify the semaphore simultaneously.

Semaphores- Example

- Consider an office with **one printer**.
- Many processes want to print documents:
 Process P1 → Print document
 Process P2 → Print document
 Process P3 → Print document
- Only **one process can use the printer at a time**.
- So we create a semaphore: Semaphore Printer = 1
- The value 1 means: One printer is available. No process is using the printer.

1. Process P1 wants to print

- It executes: wait(Printer)
- Operation: $\text{Printer} = \text{Printer} - 1$
 $\text{Printer} = 1 - 1$
 $\text{Printer} = 0$
- Now P1 enters the critical section.

2. Process P2 wants to print at the same time

- P2 executes: wait(Printer)
- Current value: $\text{Printer} = 0$

Since the printer is already occupied:

P2 must wait

It cannot enter.

P1 → Using Printer P2 → Waiting P3 → Waiting

3. P1 finishes printing

P1 executes: signal(Printer)

Operation: $\text{Printer} = \text{Printer} + 1$

$\text{Printer} = 0 + 1$

$\text{Printer} = 1$

The printer becomes available.

4. P2 can now proceed

P2 executes:

wait(Printer)

$\text{Printer} = 1 - 1$

$\text{Printer} = 0$

5. P2 enters and prints.

Now $\text{printer} = \text{printer} + 1 = 1$ (available for other processes)

Semaphores

This is how a semaphore prevents two processes from incorrectly using the same resource at the same time.

- Here, the semaphore controls access to the shared variable.
- There are mainly two types of semaphores:
 1. binary semaphore
 2. counting semaphore.

The flow is:

Initial $S = 1$

P1 calls wait()



S becomes 0



P1 uses resource



P2 calls wait()



S is 0



P2 waits



P1 calls signal()



Resource becomes available



P2 continues

Semaphores

1. Binary Semaphore:

- A **binary semaphore** usually has only two values: 0 or 1 used for mutual exclusion
- 1 → permission available
- 0 → permission not available

2. Counting Semaphore

A **counting semaphore** is used when several identical resources are available.

- Suppose a computer laboratory has 3 printers.
- We can initialize: `printer = 3`
- Now: P1 takes one printer, `printer = 2`
P2 takes one printer, `printer = 1`
P3 takes one printer, `printer = 0`
- If P4 requests a printer: `printer = 0`
P4 must wait
- When P2 finishes: `signal(printer)`
- a printer becomes available, and P4 can use it.

Semaphores

- This is why counting semaphores are useful for controlling a group of similar resources.
- The wait() operation checks whether the resource is available.

```
wait(S)
```

```
{
```

```
    if resource is available
```

```
        take permission;
```

```
    else
```

```
        wait;
```

```
}
```

- The signal() operation releases the resource.

Synchronization Problems and Semaphore-Based Solutions

- Semaphores are powerful tools for solving process synchronization problems, but designing correct semaphore solutions is challenging. Improper use of semaphores can lead to problems such as **deadlock, starvation, and inefficient resource utilization**.
- Some classical synchronization problems used to understand semaphore-based solutions are:
 1. **Bounded Buffer (Producer–Consumer) Problem**
 2. **Readers–Writers Problem**
 3. **Dining Philosophers Problem**

Dining Philosophers Problem

- The Dining Philosophers Problem is a classical synchronization problem that explains how multiple processes compete for limited shared resources and how improper resource allocation can lead to **deadlock**.
- Problem Consider **five philosophers** sitting around a circular table. Between every two philosophers, there is one fork.
 - ✓ P0, P1, P2, P3, P4 → Philosophers (Processes)
 - ✓ F0, F1, F2, F3, F4 → Forks (Resources)
- Each philosopher continuously performs two actions:

Thinking → Hungry → Eating → Thinking

1. Thinking - The philosopher does not need any fork. P0 is thinking

2. Hungry - The philosopher wants to eat. To eat, the philosopher requires:

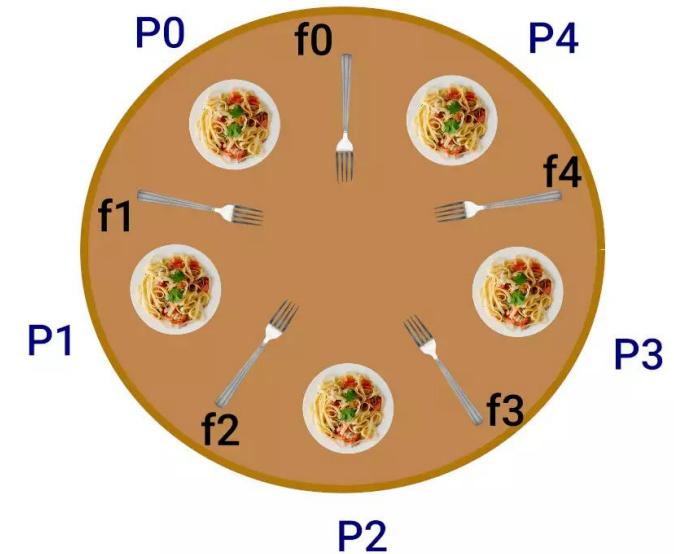
Left fork + Right fork for P0 Left fork = F0 Right fork = F1

3. Eating Only after acquiring both forks: F0 + F1 the philosopher can eat.

- After eating, both forks are released.

Challenge :

Design a synchronization mechanism so that philosophers can eat without causing **deadlock** (all waiting forever) or **starvation** (some never get a chance to eat).



Semaphore Solution to Dining Philosopher

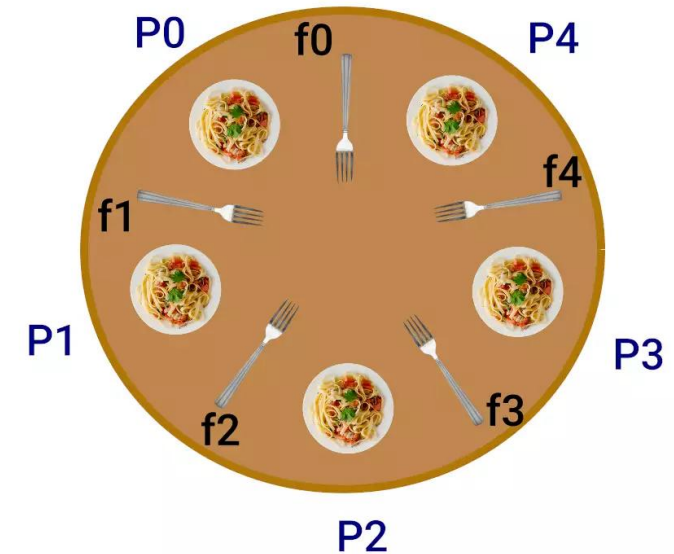
Algorithm:

- Since forks are shared resources, we use semaphores.
- Each Fork is represented as a binary semaphore (mutex).
Semaphore fork[5]
fork[0] = 1
fork[1] = 1
fork[2] = 1
fork[3] = 1
fork[4] = 1

1 → Fork is available

0 → Fork is unavailable
- Philosopher must acquire both left and right semaphores before eating.

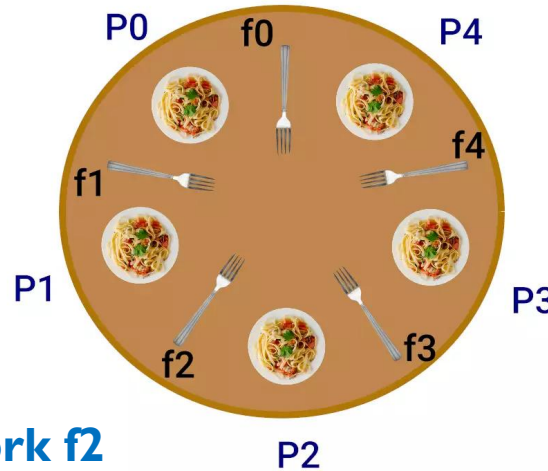
```
while(true)
{
    Think();
    wait(left fork);
    wait(right fork);
    Eat();
    signal(right fork);
    signal(left fork);
}
```
- After eating, the philosopher releases both semaphores.



Semaphore Solution to Dining Philosopher

Step 1: Philosopher P0 takes left fork f1

- P0 executes: wait(f1)
- Semaphore operation: $f1 = 1 - 1$; $f1 = 0$
- Current state: P0 holds f1
- Fork status: $f0 = 1$
 $f1 = 0$
 $f2 = 1$
 $f3 = 1$
 $f4 = 1$



Step 2: Philosopher P1 takes left fork f2

- P1 executes: wait(f2)
- After execution: $f2 = 1 - 1$; $f2 = 0$
- State: P0 → holds f1
P1 → holds f2
- Fork status: $f0 = 1$
 $f1 = 0$
 $f2 = 0$
 $f3 = 1$
 $f4 = 1$

Step 3: Philosopher P2 takes left fork f3

- P2 executes: wait(f3)
- After execution: $f3 = 0$
- State: P0 → f1 ; P1 → f2 ; P2 → f3
- Fork status: $f0 = 1$
 $f1 = 0$
 $f2 = 0$
 $f3 = 0$
 $f4 = 1$

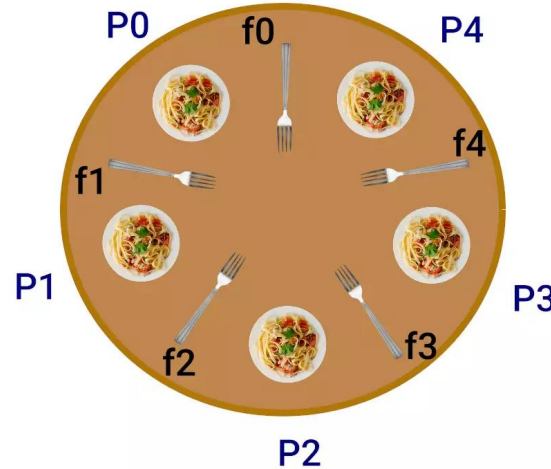
Step 4: Philosopher P3 takes left fork f4

- P3 executes: wait(f4)
- After execution: $f4 = 0$
- State: P0 → f1 ; P1 → f2 ; P2 → f3 ; P3 → f4
- Fork status: $f0 = 1$
 $f1 = 0$
 $f2 = 0$
 $f3 = 0$
 $f4 = 0$

Semaphore Solution to Dining Philosopher

Step 5: Philosopher P4 takes left fork f0

- P4 executes: wait(f0)
- After execution: $f0 = 0$
- Now:
 - P0 holds f1
 - P1 holds f2
 - P2 holds f3
 - P3 holds f4
 - P4 holds f0
- All forks are occupied.



Step 6: Now Each Philosopher Requests Second Fork

6.1 P0 requests f0

- P0 already has: f1 ✓
- Needs: f0
- But: f0 is held by P4
- Therefore: P0 waits

Final Waiting State

The dependency becomes:

P0 waiting for f0 → held by P4

P4 waiting for f4 → held by P3

P3 waiting for f3 → held by P2

P2 waiting for f2 → held by P1

P1 waiting for f1 → held by P0

No philosopher can proceed.

Result: Deadlock

The system reaches a deadlock because:

1. **Mutual Exclusion** One fork can be used by only one philosopher.
2. **Hold and Wait** Each philosopher holds one fork and waits for another.
3. **No Preemption** A fork cannot be forcibly taken away.
4. **Circular Wait** Each philosopher waits for the next philosopher.

Hence: All philosophers are waiting forever.

Semaphore Solution to Dining Philosopher

- The problem is not because of semaphores. The problem is the **way resources are requested**.
- Different semaphore-based solutions can prevent deadlock.
- The solution is to use **additional semaphore control** to ensure that the system never reaches the deadlock state.

Solution : Limit the Number of Philosophers Using a Semaphore and Now introduce an additional semaphore:

Semaphore room = 4; // Only 4 philosophers are allowed to try picking forks at the same time.

Allowed: P0 P1 P2 P3 ;Waiting: P4

Step 1: Philosophers enter the room

- Each philosopher first executes: wait(room)
- This decreases the room semaphore.

P0 enters: room = 4 - 1;
room = 3

P1 enters: room = 3 - 1
room = 2

P2 enters: room = 2 - 1
room = 1

P3 enters: room = 1 - 1
room = 0

Now: P0, P1, P2, P3 are allowed.

Semaphore Solution to Dining Philosopher

Step 1: Philosophers enter the room

- Each philosopher first executes: `wait(room)`
- This decreases the room semaphore.

P0 enters: `room = 4 - 1;`
`room = 3`

P1 enters: `room = 3 - 1`
`room = 2`

P2 enters: `room = 2 - 1`
`room = 1`

P3 enters: `room = 1 - 1`
`room = 0`

Now: P0, P1, P2, P3 are allowed.

- P4 must wait.

Step 2: Philosophers pick forks

Assume the same situation:

P0 takes f1 : `wait(f1)`
`f1 = 0`

P1 takes f2: `wait(f2)`
`f2 = 0`

P2 takes f3: `wait(f3)`
`f3 = 0`

P3 takes f4: `wait(f4)`
`f4 = 0`

Current state: P0 → holds f1
P1 → holds f2
P2 → holds f3
P3 → holds f4

Step 3: Request second forks

Now:

P0 needs f0 f0 is available

So: `wait(f0)`

P0 gets: f1 + f0

and starts eating.

Other philosophers wait temporarily

For example:

P1 needs f1.

But: f1 is with P0

So P1 waits.

Semaphore Solution to Dining Philosopher

Step 4: P0 finishes eating

- After eating: `signal(f1)`
`signal(f0)`

Forks become available.

- Also: `signal(room)` because P0 leaves the room.
- Now: `room = 1`

A waiting philosopher can enter.

Step 5: Other philosophers proceed

Now, P1 can acquire: `f1` and eat.

- The same process continues.

Why Deadlock Cannot Occur?

The original deadlock was:

P0 waits for P1

P1 waits for P2

P2 waits for P3

P3 waits for P4

P4 waits for P0

The circle was created because **all five philosophers were holding one fork.**

With: `room = 4`

maximum four philosophers can hold forks.

So the worst case becomes:

P0 holds fork

P1 holds fork

P2 holds fork

P3 holds fork

P4 is outside

There are still **5 forks but only 4 competing philosophers.**

At least one philosopher will always be able to get two forks.

Readers writer Problem

The **Readers–Writers Problem** is a classical **process synchronization problem**

- It occurs when several processes or threads access the same **shared data**, such as a file, database, or memory location.
- There are two types of processes:
 - ✓ **Reader** – only reads the shared data.
 - ✓ **Writer** – modifies/writes the shared data.
- The synchronization requirement is:
 1. Multiple readers can access data together if no writer is writing.
 2. Writers have exclusive access no other reader or writer can enter during writing

Variants of the Problem

1. Readers Preference

- Readers are given priority.
- No reader waits if the resource is available for reading, even if a writer is waiting.
- Writers may suffer from *starvation*.

2. Writers Preference

- Writers are prioritized over readers.
- Ensures writers won't starve, but readers may wait longer.

Readers writer Problem

Solution When Reader Has The Priority Over Writer

- Here priority means, no reader should wait if the shared resource is currently open for reading.
- There are four types of cases that could happen here.

Case	Process 1	Process 2	Allowed/Not Allowed
Case 1	Writing	Writing	Not Allowed
Case 2	Writing	Reading	Not Allowed
Case 3	Reading	Writing	Not Allowed
Case 4	Reading	Reading	Allowed

Synchronization Variables

- **mutex**: Ensures mutual exclusion when updating the reader count (readcnt).
- **wrt**: Controls access to the shared data (used by both readers and writers).
- **readcnt**: Number of readers currently in the critical section.

Semaphore Operations

- **wait()**: Decreases semaphore value, blocks if < 0 .
- **signal()**: Increases semaphore value, wakes waiting process if any.

Readers writer Problem

Reader Process(Reader Preference)

1. Reader requests the entry to critical section.
2. If allowed:
 - it increments the count of number of readers inside the critical section. If this reader is the first reader entering, it locks the **wrt** semaphore to restrict the entry of writers if any reader is inside.
 - It then, signals **mutex** as any other reader is allowed to enter while others are already reading.
 - After performing reading, it exits the critical section. When exiting, it checks if no more reader is inside, it signals the semaphore "wrt" as now, writer can enter the critical section.
3. If not allowed, it keeps on waiting.

```
do {  
    wait(mutex); // Lock before updating readcnt  
    readcnt++;  
    if (readcnt == 1)  
        wait(wrt); // First reader blocks writers  
    signal(mutex); // Allow other readers in  
    // ---- Critical Section (Reading) ----  
    wait(mutex);  
    readcnt--;  
    if (readcnt == 0)  
        signal(wrt); // Last reader allows writers  
    signal(mutex); // Unlock  
} while(true);
```

The first reader blocks writers, the last reader allows writers, and all readers in between share the resource. This gives preference to readers, but writers may starve.

Readers writer Problem

Writer's Process

```
do {  
    wait(wrt); // Lock resource  
  
    // ---- Critical Section (Writing) ----  
  
    signal(wrt); // Release resource  
} while(true);
```

wait(wrt): Writer locks the resource to get exclusive access.

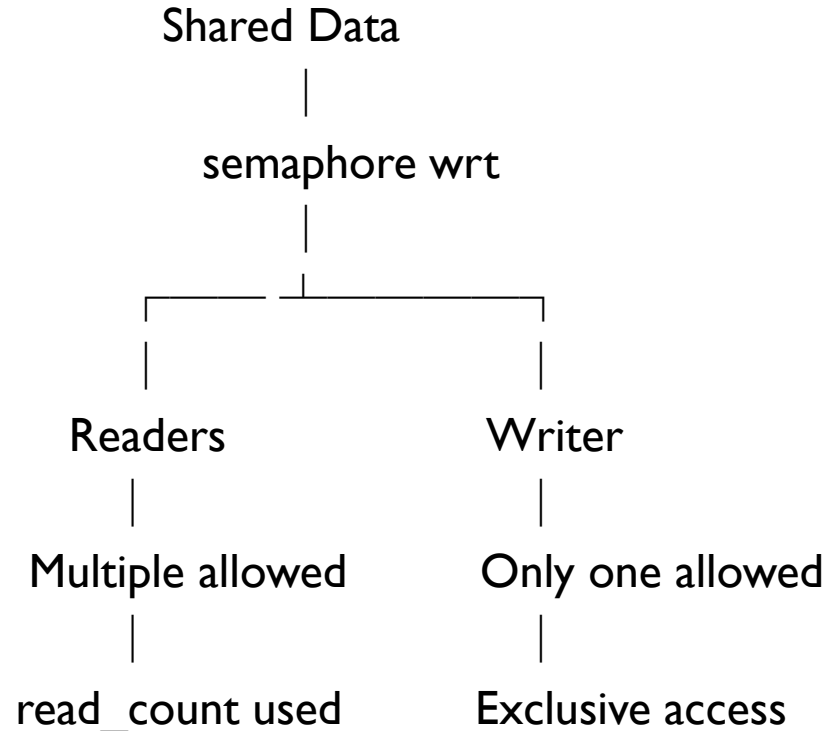
Critical Section: Writer performs writing (no other reader or writer can enter).

signal(wrt): Writer releases the resource after finishing.

- Only one writer can write at a time, and no readers are allowed while writing. This ensures data integrity.
- The Readers-Writers Problem highlights the need for proper synchronization when multiple processes access shared data.
- Readers Preference solution allows many readers to access simultaneously while blocking writers, improving read efficiency.
- However, writers may starve if readers keep arriving continuously.

To avoid this, Writers Preference or a Fair solution (using queues or advanced semaphores) can be used to ensure no starvation and balanced access.

Readers writer Problem



- Only one writer can write at a time, and no readers are allowed while writing. This ensures data integrity.
- The Readers-Writers Problem highlights the need for proper synchronization when multiple processes access shared data.
- Readers Preference solution allows many readers to access simultaneously while blocking writers, improving read efficiency.
- However, writers may starve if readers keep arriving continuously.

To avoid this, Writers Preference or a Fair solution (using queues or advanced semaphores) can be used to ensure no starvation and balanced access.

Monitors

- A **monitor** is a **high-level synchronization mechanism** used to control access to shared data among multiple processes or threads.
- The main idea is simple: A monitor allows only **one process or thread at a time** to execute inside the monitor.
- So, mutual exclusion is built into the monitor automatically.
- This is different from semaphores, where the programmer must explicitly use operations such as:

`wait(mutex);`

`...`

`signal(mutex);`

- With a monitor, the synchronization is handled as part of the monitor structure itself.

Simple Example

Suppose two processes are accessing a shared bank balance.

Shared Balance = 1000

P1 → Deposit 500

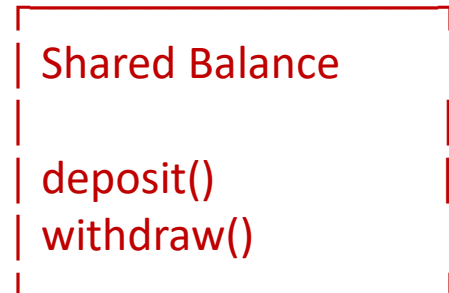
P2 → Withdraw 300

If both access the balance simultaneously, a race condition may occur.

A monitor can protect the shared balance:

Only one process can execute `deposit()` or `withdraw()` inside the monitor at a time.

Monitor



Monitors

Simple Example

Suppose two processes are accessing a shared bank balance.

Shared Balance = 1000

P1 → Deposit 500

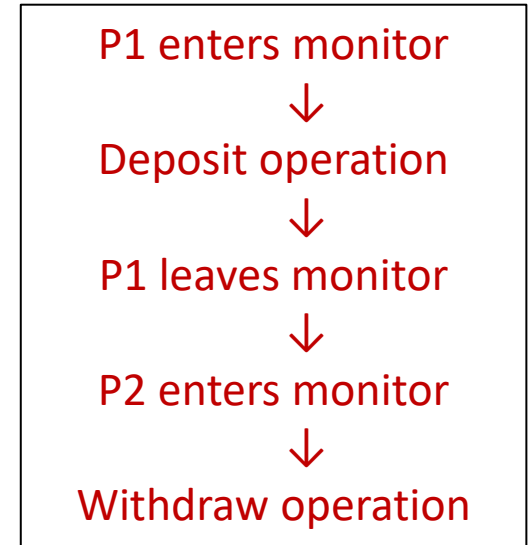
P2 → Withdraw 300

If both access the balance simultaneously, a race condition may occur.

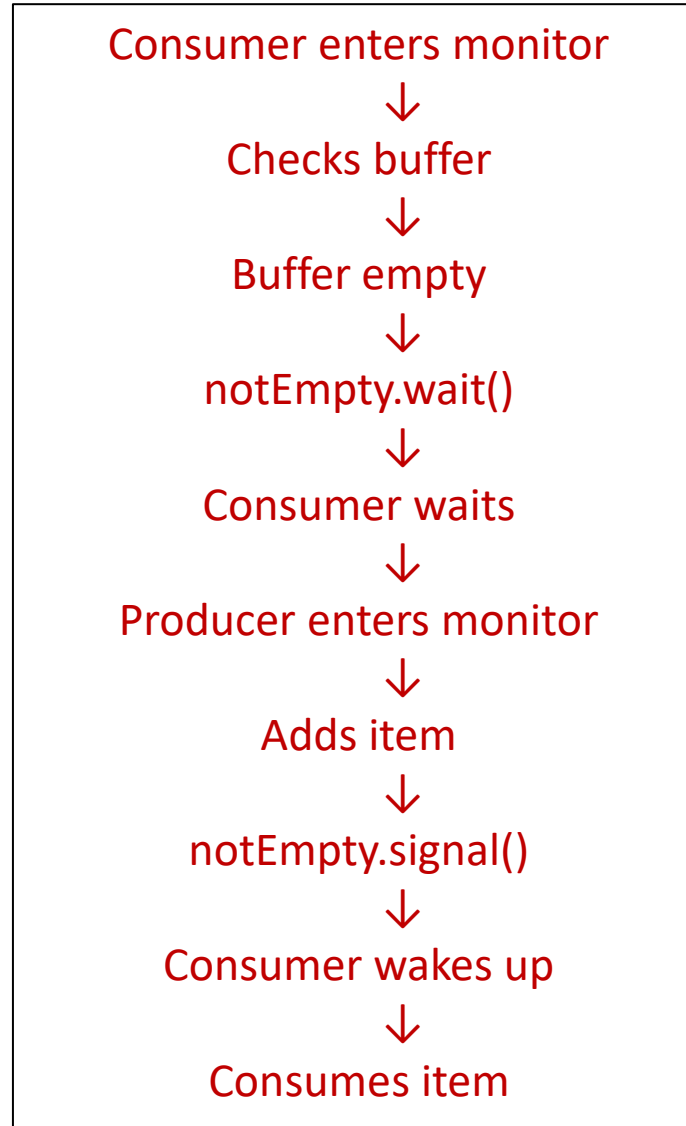
A monitor can protect the shared balance:

Only one process can execute deposit() or withdraw() inside the monitor at a time.

Therefore, the shared data is protected automatically.



Monitors – solution to Producer Consumer Problem



Monitors – solution to Dining Philosophers Problem

A **monitor** provides a clean solution to the **Dining Philosophers problem** by controlling access to the forks inside one synchronized module.

Assume five philosophers P_0, P_1, P_2, P_3, P_4 . Each philosopher can be in one of three states:

- THINKING
- HUNGRY
- EATING

A philosopher is allowed to enter the EATING state only when **both adjacent philosophers are not eating**. This indirectly guarantees that both required forks are available.

Each philosopher executes:

```
while(true)
{
    think();
    DiningPhilosophers.pickup(i);
    eat();
    DiningPhilosophers.putdown(i);
}
```

Monitors – solution to Dining Philosophers Problem

The important point is that the philosopher does **not directly pick up one fork and then wait for another**. Instead, the philosopher asks the monitor:

"Are both of my forks available?"

If **yes**, the **monitor allows** the **philosopher to eat**. If **not**, the **philosopher waits**.

For example, suppose **P0 wants to eat**. Its neighbours are **P4 and P1**.

The monitor checks:

Is P4 eating? No

Is P1 eating? No

Is P0 hungry? Yes

Therefore: P0 → EATING

Now suppose **P1 becomes hungry**. The monitor checks P1's neighbours:

P0 → EATING

P2 → THINKING

Since P0 is already eating:

P1 cannot eat

P1 → waits

At the same time, **P2 may be allowed to eat**, because P2 does not share a fork with P0.

Monitors – solution to Dining Philosophers Problem

So a possible state is:

P0 → EATING

P1 → WAITING

P2 → EATING

P3 → THINKING

P4 → WAITING

This shows an important advantage: **non-adjacent philosophers can eat simultaneously.**

When P0 finishes, it calls:

putdown(0)

The monitor changes: P0 → THINKING

and checks whether its neighbours can now eat:

test(P4)

test(P1)

If P1's other neighbour P2 is still eating, P1 continues waiting. Once both neighbours of P1 are not eating, the monitor signals P1: P1 → EATING

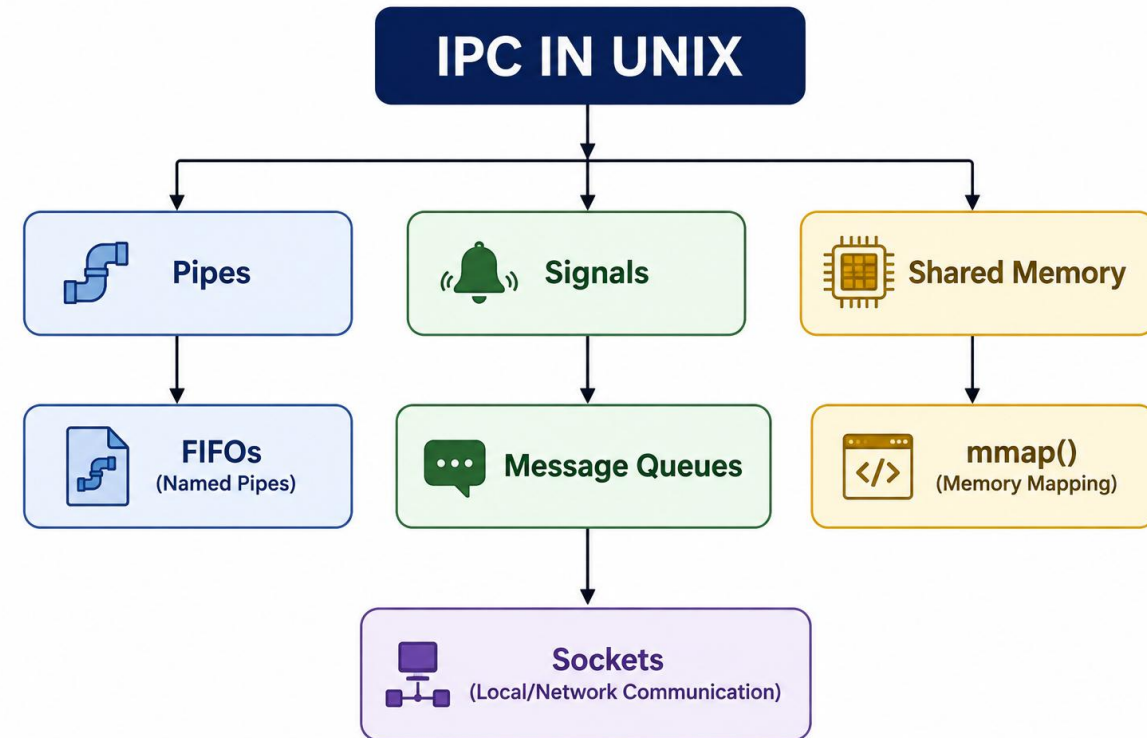
monitor prevent deadlock

Monitors – solution to Producer Consumer Problem

Semaphore	Monitor
Low-level synchronization mechanism	High-level synchronization mechanism
Programmer explicitly uses wait() and signal()	Mutual exclusion is built in
Easier to make synchronization mistakes	Generally easier and safer to use
Uses semaphore variables	Uses shared data, procedures, and condition variables
Programmer controls locking manually	Monitor controls access automatically

IPC in UNIX

- **IPC (Inter-Process Communication)** is a set of mechanisms provided by the operating system that allows **independent processes to communicate with each other and synchronize their activities**.
- Normally, each process has its own separate memory space. A process cannot directly access another process's memory.
- Therefore, UNIX provides IPC mechanisms to allow controlled communication
- IPC is mainly used for:
 1. Sharing data between processes
 2. Synchronizing process execution
 3. Implementing client-server applications
 4. Coordinating parallel processes



Key idea: Different IPC mechanisms are suitable for different communication requirements.

IPC in UNIX

- **Pipes and FIFOs:** **Anonymous pipes** (`pipe()`) allow unidirectional data flow between a parent and child process. Managed by the kernel. **FIFOs (named pipes)** extend this to allow unrelated processes to communicate via the file system.
- **Message Queues:** Allow processes to exchange **data as discrete packets or messages** managed by the kernel, supporting **asynchronous** communication.
- **Shared Memory:** Maps a **common memory segment** into the address space of multiple processes. It offers the **fastest data exchange** because it avoids kernel copying overhead, but it requires synchronization tools like semaphores.
- **Semaphores:** **Integer variables used as synchronization primitives** to control concurrent access to shared resources and prevent race conditions.
- **Signals:** Lightweight **asynchronous notifications** sent to a process **to signal events or interrupt execution** (such as SIGINT or SIGTERM).
- **Sockets:** Enable **bidirectional communication** between processes on the **same machine** (Unix domain sockets) or **across a network** (Internet sockets).

IPC in UNIX

- **PIPES:**

- A **pipe** provides a communication channel through which one process can send a stream of data to another. Data flows sequentially FIFO

- Basic operations:

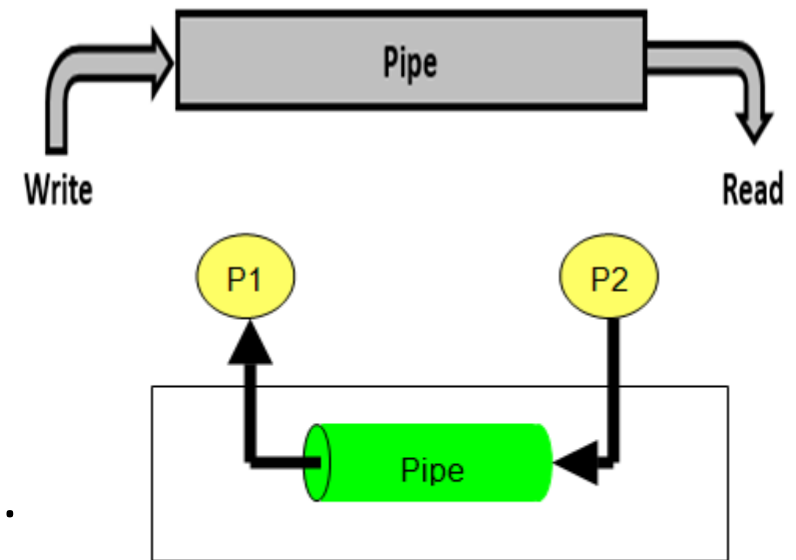
fd[0] → Read end of pipe

fd[1] → Write end

- pipe(fd);
- write(fd[1], data, size);
- read(fd[0], buffer, size);

- **FIFO — Named Pipe :**

- A **FIFO** is a named pipe represented in the file system.
- Process A —► Named FIFO —► Process B
- **Difference:** Ordinary pipes are commonly used between **related processes**; FIFOs provide a **named IPC endpoint usable by unrelated processes** subject to permissions.



IPC in UNIX

- **Signals**

- A **signal** is used to notify a process that an event has occurred.
 - Examples:
 - SIGINT → Interrupt
 - SIGTERM → Termination request
 - SIGKILL → Forced termination
 - SIGCHLD → Child status change
 - Signals are mainly used for **event notification**, not bulk data transfer.
-
- **Message Queues** - Processes communicate using **discrete messages** stored in a queue. The processes do not communicate directly. The kernel maintains a queue. Asynchronous communication, Messages can have priorities, Processes do not need to execute at the same time
 - **Shared Memory** - Two or more processes access a common memory region. Very fast, No repeated copying of data, Suitable for large data exchange
-
- **mmap()** - mmap() can map files or shared-memory objects into a process's virtual address space and can be used for shared mappings.

IPC in UNIX

- **Sockets** Sockets are used for communication
 - Between processes on the same machine
 - Between processes on different machines
 - Examples: Web browsers communicating with servers , Database connections
 - Distributed applications
 - Types: **UNIX Domain Socket** Communication within the same computer

Network Socket Communication over a network

IPC Mechanism	Main Purpose	Data Transfer
Pipes	Simple process communication	Stream of bytes
FIFO	Communication between unrelated processes	Stream of bytes
Signals	Event notification	Small notification
Message Queue	Exchange structured messages	Messages
Shared Memory	Fast data sharing	Large data
mmap()	Memory-based file sharing	Memory region
Sockets	Local/network communication	Data packets

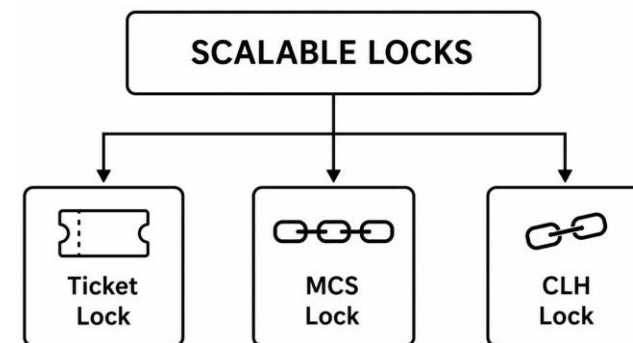
Multiprocessors and Locking

- A **multiprocessor/multicore system** allows multiple threads to execute simultaneously on different CPU cores.
- Locks provide - **Mutual exclusion when multiple threads access shared data.**
 - **Problem** - On many cores, several threads may compete for the **same lock** - This is called **Lock Contention**
- **Why Simple Locks Do Not Scale Well**
 - Suppose many threads compete for one lock.
 - Only one thread acquires the lock.
 - Others may **spin/wait**.
- **Cache-Coherence Effect**
 - Different CPU cores have caches.
 - Repeated atomic modifications to the same lock cache line cause **coherence traffic/cache-line bouncing**.
 - **Important:** A lock can be logically correct but still have poor performance.

Multiprocessors and Locking

- **SCALABLE LOCKS:**

- A **scalable lock** is designed to reduce contention and maintain better performance as the number of competing processors/threads increases.



- **Ticket Lock** - A **Ticket Lock** works like taking a token at a service counter.
- **Queue-Based Locks** - Instead of all threads continuously competing on the same lock variable, threads are organized into a queue.
 - **MCS Lock (Mellor-Crummey and Scott Lock)**
 - Each waiting thread has a queue node.
 - Threads form a queue.
 - Each thread primarily spins on its **own/local node state**.
 - Good scalability under high contention.
 - **CLH Lock**
 - A thread waits based on its **predecessor's state**.
 - Reduces contention compared with a simple centralized spinlock.

Lock-Free Coordination

- Locks can cause
 - Blocking / Spinning + Contention + Possible Deadlock in multi-lock designs
- Why do we want to avoid locks?
 - performance
 - complexity
 - deadlock
- **Lock-Free Coordination**
 - Threads coordinate using **atomic operations** without requiring conventional mutual-exclusion lock ownership for the operation.
 - Uses CAS (Compare-and-Swap)

Synchronization Examples

- Activity : Explore how synchronization is implemented in
 - Solaris
 - Windows XP
 - Linux
 - Pthreads

Solaris Synchronization

- Implements a variety of locks to support multitasking, multithreading (including real-time threads), and multiprocessing
- Uses **adaptive mutexes** for efficiency when protecting data from short code segments
 - Starts as a standard semaphore **spin-lock**
 - If lock held, and by a thread running on another CPU, spins
 - If lock held by non-run-state thread, block and sleep waiting for signal of lock being released
- Uses **condition variables**
- Uses **readers-writers** locks when longer sections of code need access to data
- Uses **turnstiles** to order the list of threads waiting to acquire either an adaptive mutex or reader-writer lock
 - Turnstiles are per-lock-holding-thread, not per-object
- **Priority-inheritance per-turnstile gives the running thread the** highest of the

Windows XP Synchronization

- Uses interrupt masks to protect access to global resources on uniprocessor systems
- Uses **spinlocks** on multiprocessor systems
 - Spinlocking-thread will never be preempted
- Also provides **dispatcher objects** user-land which may act mutexes, semaphores, events, and timers
 - **Events**
 - An event acts much like a condition variable
 - **Timers** - notify one or more thread when time expired
 - Dispatcher objects either **signaled-state** (object available) or **non-signaled state** (thread will block)

Linux Synchronization

- Linux:
 - Prior to kernel Version 2.6, **disables interrupts** to implement short critical sections
 - Version 2.6 and later, **fully preemptive**
- Linux provides:
 - semaphores
 - spinlocks
 - reader-writer versions of both
- On single-CPU system, **spinlocks replaced by enabling and disabling kernel preemption**

Pthreads Synchronization

- Pthreads API is OS-independent
- It provides:
 - mutex locks
 - condition variables
- Non-portable extensions include:
 - read-write locks
 - spinlocks